

Project 4: Fundamentals of Computer Vision

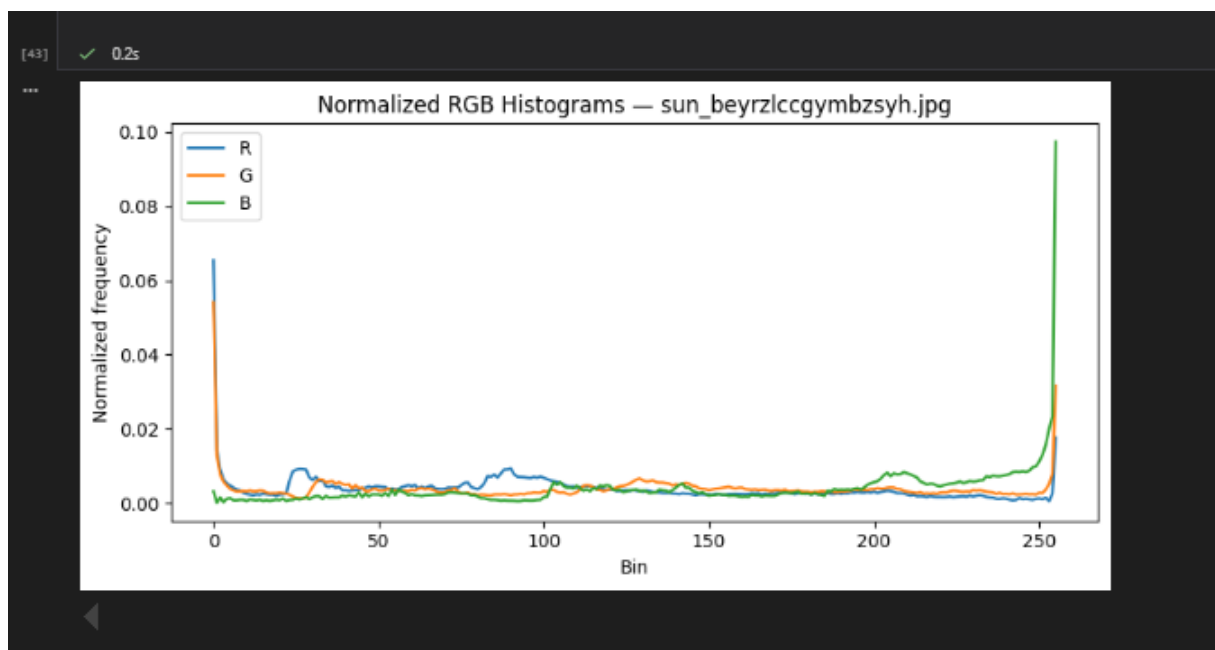
Answer the questions briefly and directly, including the images obtained from the execution of your code to explain your conclusions. Answers must be connected to the executed code.

FEATURE DESCRIPTORS: (Normalized Color Histograms and Color Space Transformations)

1. What is the purpose of normalizing histograms, and how does this technique would help in feature extraction for image processing tasks?

The purpose of **histogram normalization** is to make feature descriptors independent of the image size and the total number of pixels. In the implemented code, each histogram is normalized so that the sum of all bins equals 1. This allows fair comparisons between images of different resolutions or scales.

By normalizing histograms, the feature descriptor represents **relative frequency distributions of intensity values**, rather than absolute counts. This is essential in feature extraction because similarity metrics (such as histogram correlation used in this project) assume comparable distributions. Without normalization, larger images would dominate the comparison simply due to having more pixels, leading to misleading similarity results.



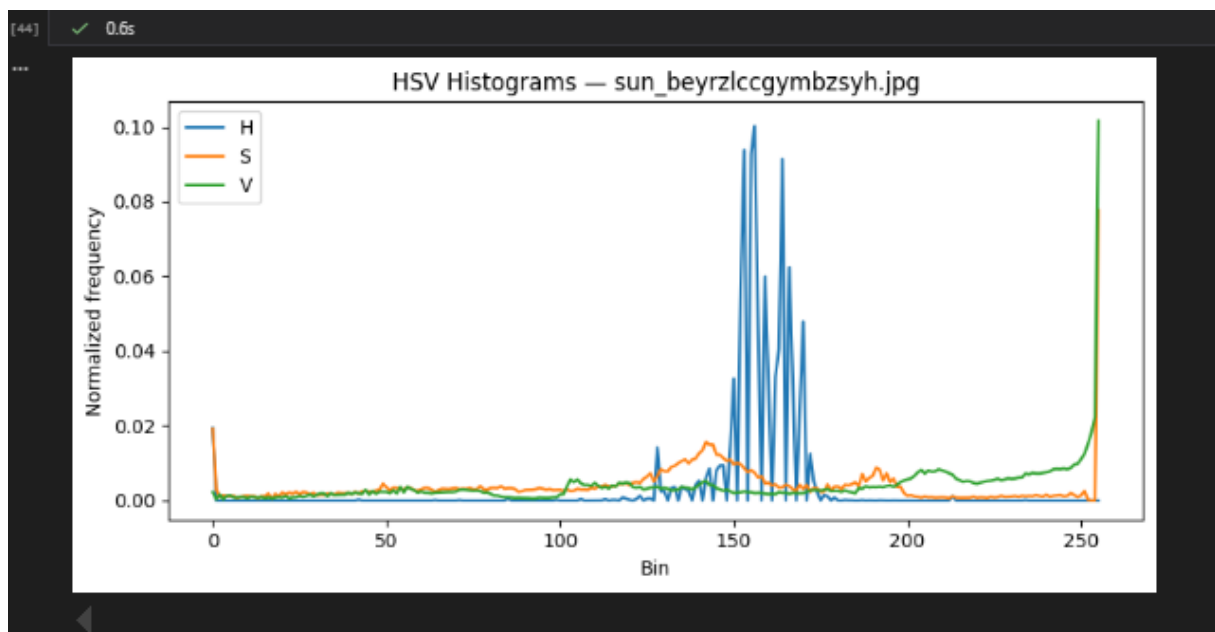
2. After transforming an image to HSV, LAB, and YCrCb color spaces, how does the interpretation of intensity values in each color space change compared to the RGB color space? Visualize the histograms of each space.

In the RGB color space, each channel (Red, Green, Blue) mixes color and illumination information. As a result, changes in lighting conditions directly affect all channels, making RGB histograms sensitive to brightness variations.

After transforming the image to other color spaces:

- **HSV:**

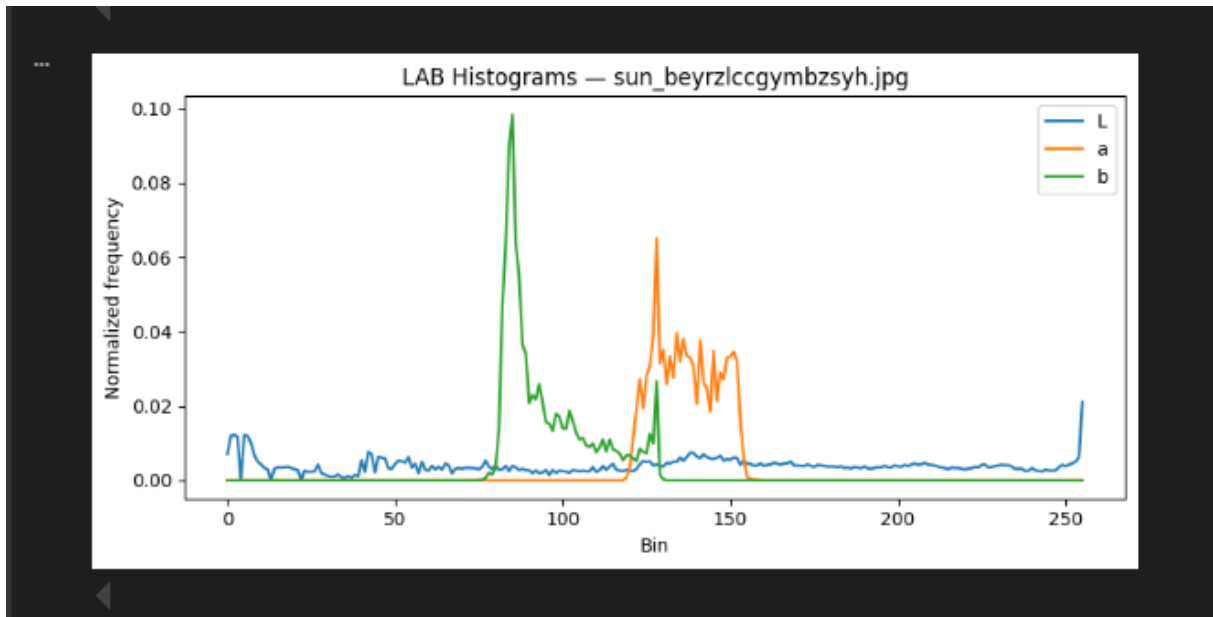
The Hue channel represents color information independently of intensity, while Saturation and Value capture color purity and brightness. This separation makes HSV histograms more robust to illumination changes.



- **LAB:**

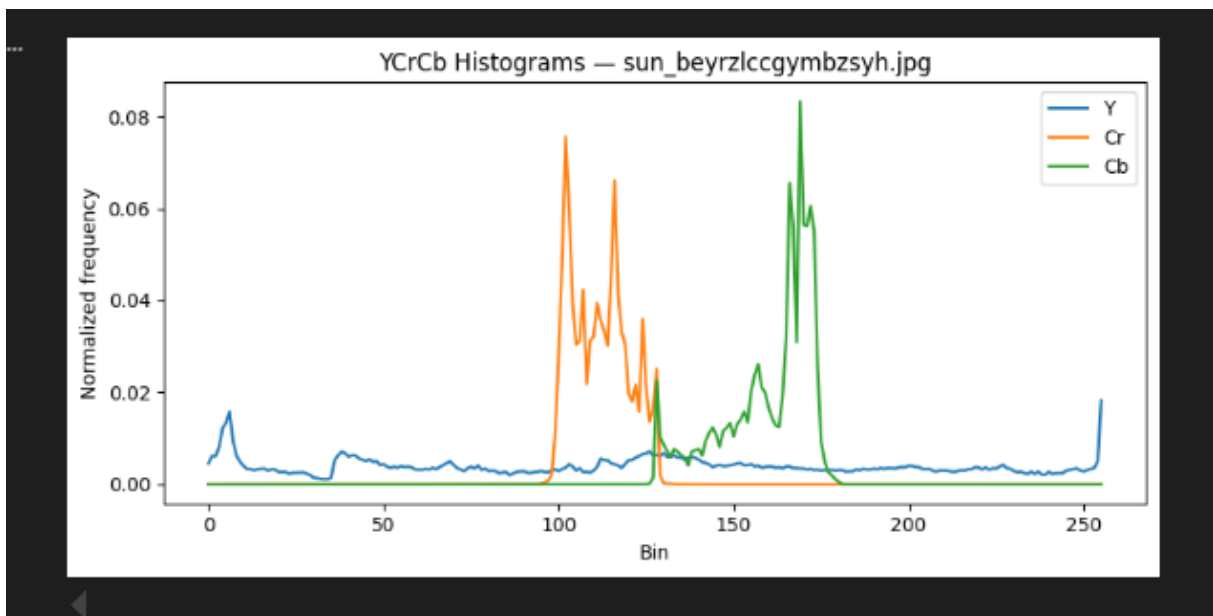
The L channel represents luminance, while the a and b channels encode chromatic information. LAB is perceptually uniform, meaning equal changes in values correspond to similar perceptual differences. This improves

interpretability and robustness in histogram-based comparisons.



- **YCrCb:**

The Y channel represents luminance, and Cr/Cb represent chrominance. By separating brightness from color, YCrCb reduces the impact of lighting changes on color distribution.



The histogram visualizations clearly show that HSV, LAB, and YCrCb isolate color information more effectively than RGB, leading to more meaningful feature representations.

3. Apply salt-and-pepper noise to the tree1.jpg image located in the dataset_histogram folder, and save the resulting noisy image as tree1_noisy.jpg in the same folder. Select a color space (e.g., HSV, LAB, or YCrCb) and compute the histograms for all images in the dataset_histogram folder, including tree1.jpg and tree1_noisy.jpg. Finally, **compare the histograms****: evaluate the similarity between the histogram of tree1.jpg and the histograms of all other images in the dataset using a similarity metric such as correlation.**

In this experiment, **salt-and-pepper noise** was applied to the reference image selected from the dataset_histograms folder. The noisy image was saved as <reference_name>_noisy.jpg in the same folder, ensuring it was included in the dataset comparison.

After selecting a color space (HSV in the experiments), normalized histograms were computed for:

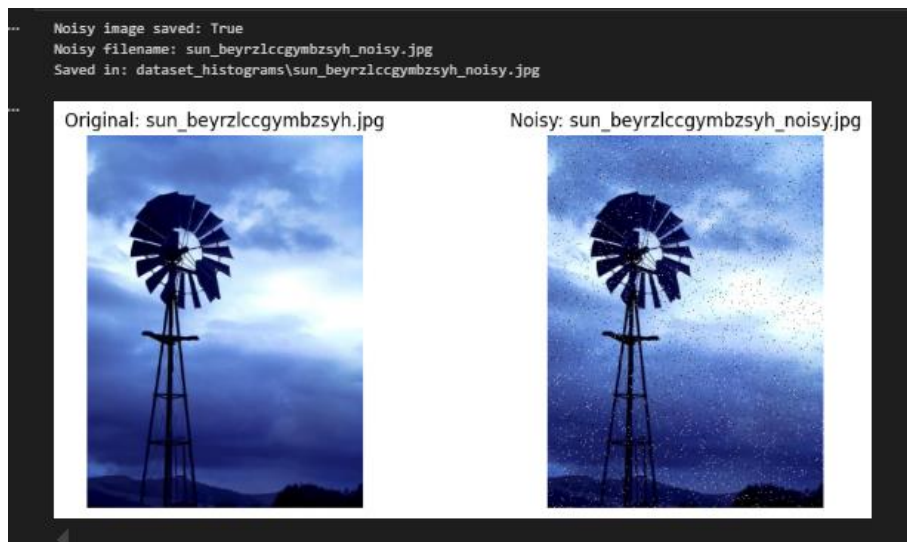
- the original reference image,
- the noisy version,
- all other images in the dataset.

Histogram similarity was evaluated using **correlation (cv2.compareHist with CORREL metric)**.

Observations:

- The noisy image achieves a **high correlation score** with the original reference image, indicating that global color histograms are relatively robust to sparse impulsive noise.
- Images with different semantic content but similar color distributions may still obtain moderate similarity scores.

- Histogram-based comparison captures global color statistics but ignores spatial information, which limits its discriminative power.



SIFT, SURF, AND ORB:

****NOTE:** I was not able to implement SURF due to a combination of time constraints and the fact surf is patented and not part of the main OpenCV so I would need to install some special packages. In some newer OpenCV versions SURF is gone completely.

4. What impact does the computational time of each feature extraction method (SIFT, SURF, ORB) have on processing larger image datasets or real-time applications? Which method is the most efficient in terms of speed?

The computational time of feature extraction when processing large datasets can create a bottleneck in real-time systems (i.e. robotics and tracking). ORB is generally known to be faster, due to its implementation of FAST and BRIEF and matching using Hamming distances. It is also known to scale well to large datasets. Strangely, in my implementation SIFT returned faster results in all cases. Usually, SIFT is slower because of its high computation cost. There are many floating point operations and the DoG extrema search can be very expensive. These factors make it not ideal for real-time applications without additional GPU acceleration. In my implementation we see that SIFT was faster than ORB, this can be explained by the fact that less keypoints are detected by SIFT than ORB. When ORB goes to the BRIEF stage it must compute many more descriptors, and having more keypoints makes drawing take much longer. SURF should theoretically be faster than SIFT but still wouldn't be as efficient as ORB for large datasets. It is a blended model between SIFT and ORB.

Number of Keypoints: Sift: 398, ORB: 1134 Speed: Sift: 3.47688889503479, ORB: 0.034180402755737305 ORB had 736 more keypoints than Sift Sift was 3.44271 seconds faster than ORB Bad SIFT sizes: [] Bad ORB sizes: []	Number of Keypoints: Sift: 435, ORB: 2710 Speed: Sift: 4.892345190048218, ORB: 0.14234566688537598 ORB had 2275 more keypoints than Sift Sift was 4.75000 seconds faster than ORB Bad SIFT sizes: [] Bad ORB sizes: []
Number of Keypoints: Sift: 97, ORB: 1870 Speed: Sift: 3.7367913722991943, ORB: 0.05928206443786621 ORB had 1773 more keypoints than Sift Sift was 3.67751 seconds faster than ORB Bad SIFT sizes: [] Bad ORB sizes: []	Number of Keypoints: Sift: 361, ORB: 971 Speed: Sift: 3.5865705013275146, ORB: 0.032210350036621094 ORB had 610 more keypoints than Sift Sift was 3.55436 seconds faster than ORB Bad SIFT sizes: [] Bad ORB sizes: []
Number of Keypoints: Sift: 402, ORB: 374 Speed: Sift: 3.7346792221069336, ORB: 0.012689590454101562 Sift had 28 more keypoints than ORB Sift was 3.72199 seconds faster than ORB Bad SIFT sizes: [] Bad ORB sizes: []	

5. How do modifications such as noise, scale, and rotation affect the performance of SIFT, SURF, and ORB in terms of keypoint detection and description?

In theory, SIFT is moderately robust to noise since we use Gaussian smoothing in the scale-space. In addition, gradient histograms average out local differences in intensities so descriptors are stable even in presence of noise. SURF tends to be more robust to noise than SIFT because by using integral images the signal is smoothed. On the other hand, the FAST component of ORB tends to be extremely sensitive to noise so false corners can be detected. The BRIEF component compares raw pixel intensities so noise can flip the bits. In effect, ORB detection quality degrades significantly in the presence of noise unless some sort of blurring is applied first.

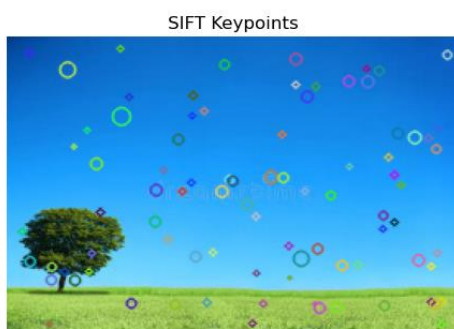
In terms of robustness to scale, SIFT is designed for scale invariance since we create a scale-space using Difference-of-Gaussians across octaves. When finding descriptors, patch sizes scale with a given keypoints sigma. Typically, SURF uses a Hessian matrix at multiple scales so it also tends to be robust to scale variance. ORB's FAST component is not scale invariant on its own, but when implementing ORB we add a pyramid that allows it to be more robust to scale changes. BRIEF is not scale invariant, so in effect ORB is not as robust to scale variance as SIFT or SURF.

When it comes to rotation invariance it really depends on how the orientation is assigned. In SIFT, orientation is computed from the gradient histogram and then the descriptors are rotated so this direction becomes angle 0. This makes it very robust to rotation. SURF is also robust to rotation variance but to a lesser extent. This is because in SURF, the Haar wavelet responses are computed and the orientation is estimated based on them. ORB is different from the other two because it uses the intensity centroid method to estimate the orientations. This works by computing the center of mass of pixel intensities in a patch and then finding a vector from the keypoint to the "centroid". The vector points in the direction of the orientation. This method is weaker because it's unstable in low-texture regions and noisy patches or when working with symmetric patterns.

Noisy:

No-Noise:

tree1.jpg



With Noise:

tree1_noisy.jpg

SIFT Keypoints



ORB Keypoints



No-Noise:

tree2.jpg

SIFT Keypoints



ORB Keypoints



With Noise:

tree2_noisy.jpg

SIFT Keypoints



ORB Keypoints



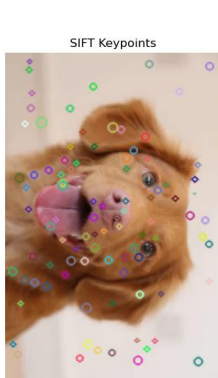
Rotated:

Original:

dog.jpg

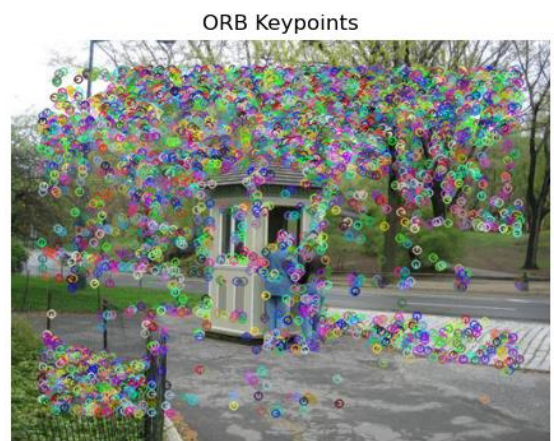


Rotated:



Scale Variance:

labelme_wgyadeanwhhmcrw.jpg

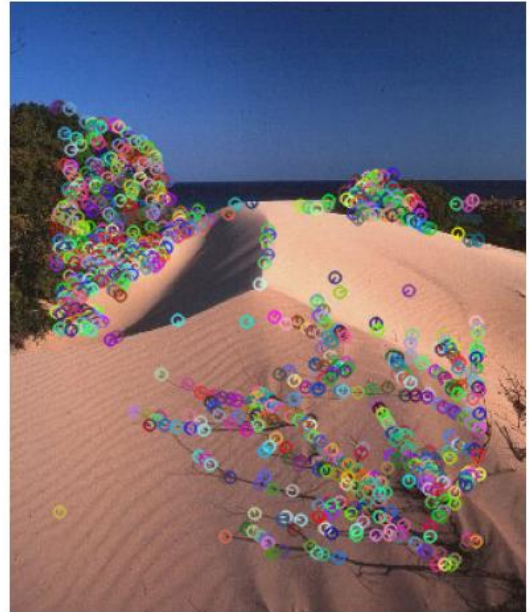


sun_atdkpuhnmiklbvzf.jpg

SIFT Keypoints



ORB Keypoints



6. How do the detected keypoints appear visually when overlaid on the image for each method (SIFT, SURF, ORB)? Are there any noticeable differences in the locations or distribution of keypoints?

SIFT keypoints usually appear as circles with orientation arrows. The size of the circles relates to the scale that was detected and the arrow is the dominant gradient direction. For SURF, keypoint circles should also come with accompanying orientation arrows. The circles tend to be larger and more uniform because SURF uses a box-filter approximation. ORB usually shows lots of small circles with no orientation arrows. There are many of them because non-max suppression is enabled and they tend to cluster at strong corners.

Interestingly, in our implementation ORB was very accurate in detecting objects and often had high numbers of keypoints centered on or around them. SIFT had a much more sparse assortment of keypoints and they were spread across the image as opposed to being concentrated on the images main features. This is likely a result of threshold setting and SIFT being more selective. It shows the difference between FAST's corner focused detection and SIFT's multi-scale blob-and-corner detection.

sun_bawsdjisukbbginm.jpg

SIFT Keypoints



ORB Keypoints



sun_bxmomsywaroyoivk.jpg

SIFT Keypoints



ORB Keypoints

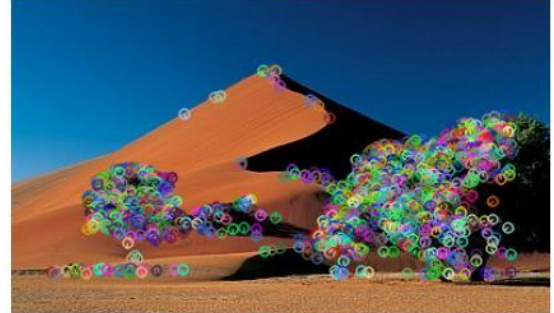


sun_adpbjcrpyetqykv.jpg

SIFT Keypoints

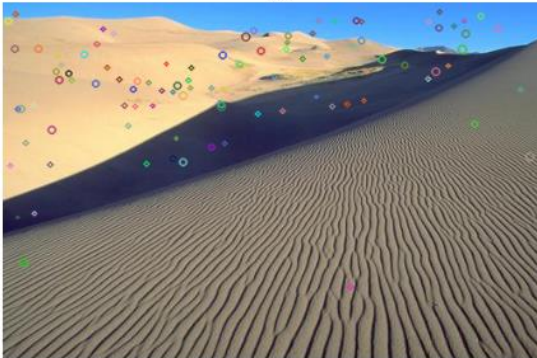


ORB Keypoints

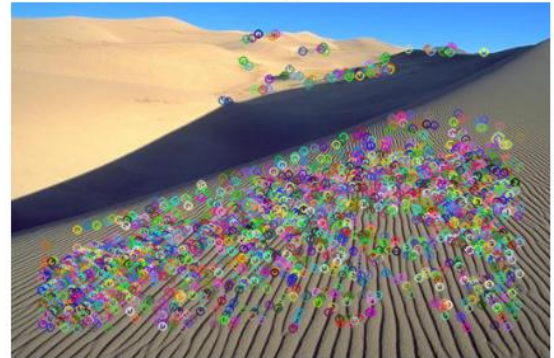


sun_asxxlgfchcyhhjgr.jpg

SIFT Keypoints



ORB Keypoints



BAG OF VISUAL WORDS:

7. What role does the SIFT algorithm play in the BoVW model, and how is it used for feature extraction?

In the Bag of Visual Words model, the SIFT algorithm is responsible for extracting local features from images. SIFT detects distinctive keypoints in an image and computes a descriptor for each keypoint that encodes local gradient information. These descriptors are invariant to scale, rotation, and small illumination changes, which makes them suitable for comparing visual patterns across different images. In the implemented code, SIFT is applied to each training and test image to generate a set of local feature

descriptors that represent the visual content of the image. These descriptors form the basic input for the next stages of the BoVW pipeline.

8. Explain the process of creating the vocabulary (codebook) in the BoVW model. Why is clustering important, and how does K-Means contribute to this process?

The visual vocabulary, also called the codebook, is created by clustering the SIFT descriptors extracted from all training images. Since each image can produce hundreds of local descriptors, clustering is necessary to group similar descriptors into a finite number of representative visual words. In this implementation, K-Means clustering is applied to all training descriptors, and each cluster center represents a visual word. Clustering is important because it transforms a large and variable set of local features into a fixed-size representation that can be used by a classifier. K-Means contributes by efficiently partitioning the descriptor space into a predefined number of clusters, allowing the model to learn common visual patterns present in the dataset.

9. How do you convert the feature descriptors of images into histograms of visual words? What is the importance of the histogram in the BoVW framework?

After the codebook is created, each SIFT descriptor of an image is assigned to the nearest visual word using the trained K-Means model. For each image, a histogram is constructed by counting how many descriptors are assigned to each visual word. This histogram represents the image as a frequency distribution of visual words. The histogram is important in the BoVW framework because it converts an image with a variable number of local features into a fixed-length feature vector. This representation allows images to be directly compared and used as input for machine learning classifiers such as SVM.

10. Why is normalization of the histograms necessary before feeding them into the classifier? What method is used for normalization in this exercise?

Histogram normalization is necessary because images may contain different numbers of detected keypoints, which would otherwise lead to histograms with different magnitudes. Without normalization, images with more features could dominate the classification process, leading to biased results. In this exercise, L2 normalization is applied to each histogram, ensuring that all feature vectors have comparable scale. This improves the stability and performance of the SVM classifier by making the decision boundary less sensitive to variations in feature count.

11. Evaluate the model on the test set by calculating precision and recall for each class, and analyze its performance using a confusion matrix.

The model was evaluated on a test set of 40 images, equally divided between the aquarium and desert classes. The obtained results show a precision, recall, and F1-score of 0.90 for both classes, indicating balanced and consistent performance. The

overall accuracy of the model is 90 percent. The confusion matrix shows that most images are correctly classified, with only a small number of misclassifications between the two classes. This indicates that the BoVW representation combined with SIFT features and a linear SVM is effective at distinguishing between aquarium and desert scenes, while maintaining good generalization on unseen data.

Train images: 160				
Test images: 40				
	precision	recall	f1-score	support
aquarium	0.90	0.90	0.90	20
desert	0.90	0.90	0.90	20
accuracy			0.90	40
macro avg	0.90	0.90	0.90	40
weighted avg	0.90	0.90	0.90	40